nsip

Numerical Sets and Integer Partitions

0.1

3 March 2023

Naci Er

Naci Er
Email: naaci@live.com

Contents

Chapter 1

Introduction

1.1 Loading Package

Example

```
gap> LoadPackage("nsip");
true
```

Chapter 2

Numerical Sets

2.1 Category of Numerical Sets

2.1.1 IsNumericalSet (for IsAttributeStoringRep)

▷ `IsNumericalSet(arg)` (filter)

Returns: `true` or `false`

A numerical set is a cofinite subset of $\mathbb{N}$ which contains 0.

2.2 Construction of A Numerical Set

2.2.1 NumericalSet (for IsList)

▷ `NumericalSet(List)` (attribute)

Returns: NumericalSet

The minimal numerical set conating the given elements.

2.2.2 NumericalSetByGaps (for IsList)

▷ `NumericalSetByGaps(List)` (attribute)

Returns: NumericalSet

Example

```
gap> S := NumericalSet([0,3,4,5,7]);
{0,3,4,5,7,->}
```

Example

```
gap> T := NumericalSetByGaps([1,2,6]);
{0,3,4,5,7,->}
```

Example

```
gap> S  = T;
true
```

2.3 Attributes of A Numerical Set

2.3.1 SmallElements (for IsNumericalSet)

▷ `SmallElements(NumericalSet)` (attribute)

Returns: List

SmallElements of *S* is the subset S which are smaller than the Conductor.

2.3.2 Gaps (for IsNumericalSet)

▷ `Gaps(Gaps)` (attribute)

Returns: List

2.3.3 Genus (for IsNumericalSet)

▷ `Genus(NumericalSet)` (attribute)

Returns: Integer

Genus of *S* is the number of the gaps of *S*.

2.3.4 Length (for IsNumericalSet)

▷ `Length(NumericalSet)` (attribute)

Returns: Integer

Length of *S* is the number of the small elements of *S*.

2.3.5 Conductor (for IsNumericalSet)

▷ `Conductor(NumericalSet)` (attribute)

Returns: Integer

The conductor of *S* is the smallest element of *S* such that every subsequent integer is an element of *S*.

2.3.6 FrobeniusNumber (for IsNumericalSet)

▷ `FrobeniusNumber(NumericalSet)` (attribute)

Returns: Integer

The largest gap of *S* is called the Frobenius number of *S*.

2.3.7 Multiplicity (for IsNumericalSet)

▷ `Multiplicity(NumericalSet)` (attribute)

Returns: Integer

2.3.8 PseudoFrobeniusNumbers (for IsNumericalSet)

▷ `PseudoFrobeniusNumbers(NumericalSet)` (attribute)

Returns: Integer List

The type of an numerical set is the number of pseudo-Frobenius numbers of the numerical set. Given a numerical set S, the set of pseudo-Frobenius numbers of S is defined by

$$PF(S) = \{x \in \mathbb{Z} \setminus S : x + S \setminus \{0\} \subseteq S\}.$$

2.3.9 GapsOfFirstType (for IsNumericalSet)

▷ `GapsOfFirstType(`*`NumericalSet`*`)` (attribute)

Returns: List

The set of gaps of the first type is

$$N(S) = \{x \in \mathbb{N} \setminus S : F(S) - x \in S\}$$

2.3.10 GapsOfSecondType (for IsNumericalSet)

▷ `GapsOfSecondType(`*`NumericalSet`*`)` (attribute)

Returns: List

The set of gaps of the second type gaps $L(S)$ consists of the remaining gap numbers, i.e., $L(S) = G(S) \setminus N(S)$.

2.3.11 Dual (for IsNumericalSet)

▷ `Dual(`*`NumericalSet`*`)` (attribute)

Returns: NumericalSet

2.3.12 Dual2 (for IsNumericalSet)

▷ `Dual2(`*`arg`*`)` (attribute)

2.3.13 Atom (for IsNumericalSet)

▷ `Atom(`*`NumericalSet`*`)` (attribute)

Returns: NumericalSet

2.3.14 IsPositiveSemiSymmetric (for IsNumericalSet)

▷ `IsPositiveSemiSymmetric(`*`NumericalSet`*`)` (property)

Returns: `true` or `false`

2.3.15 IsNegativeSemiSymmetric (for IsNumericalSet)

▷ `IsNegativeSemiSymmetric(`*`NumericalSet`*`)` (property)

Returns: `true` or `false`

Numerical monoids are always negative semisymmetric \cite{Antokoletz2002}

2.3.16 IsSemiSymmetric (for IsNumericalSet)

▷ `IsSemiSymmetric(`*`NumericalSet`*`)` (property)

Returns: `true` or `false`

2.3.17 IsSymmetric (for IsNumericalSet)

▷ `IsSymmetric(NumericalSet)` (property)

Returns: `true` or `false`

2.3.18 IsAlmostSymmetric (for IsNumericalSet)

▷ `IsAlmostSymmetric(NumericalSemigroupSet)` (property)

Returns: `true` or `false`

2.3.19 IsSuperSemiSymmetric (for IsNumericalSet)

▷ `IsSuperSemiSymmetric(NumericalSet)` (property)

Returns: `true` or `false`

Example
```
gap> SmallElements(S);
[ 0, 3, 4, 5, 7 ]
```

Example
```
gap> Genus(S);
3
```

Example
```
gap> Length(S);
4
```

Example
```
gap> FrobeniusNumber(S);
6
```

Example
```
gap> Conductor(S);
7
```

Example
```
gap> Multiplicity(S);
3
```

Example
```
gap> PseudoFrobeniusNumbers(S);
[ 6 ]
```

Example
```
gap> Type(S);
1
```

Example
```
gap> Atom(S);
{0,4,5,7,->}
```

Example
```
gap> Dual(S);
{0,4,5,7,->}
```

Example
```
gap> GapsOfFirstType(S);
[ 1, 2, 6 ]
```

Example

```
gap> GapsOfSecondType(S);
[  ]
```

Example

```
gap> IsNegativeSemiSymmetric(S);
true
```

Example

```
gap> IsPositiveSemiSymmetric(S);
false
```

Example

```
gap> IsSemiSymmetric(S);
true
```

Example

```
gap> IsSymmetric(S);
false
```

Example

```
gap> IsAlmostSymmetric(S);
true
```

2.4 Numerical Semigroups

2.4.1 IsNumericalSemigroupSet (for IsNumericalSet)

▷ `IsNumericalSemigroupSet(`*`NumericalSet`*`)` (property)

Returns: `true` or `false`

Chapter 3

Integer Partitions

3.1 Construction of An Integer Partition

3.1.1 IntegerPartition (for IsList)

▷ `IntegerPartition(Parts)` (attribute)

Returns: IntegerPartition

Example
```
gap> P := IntegerPartition([7,2,1,1]);
SG   A + 4 2 11= 7+ 2+ 1+ 1
```

3.2 Attributes of An Integer Partition

3.2.1 Total (for IsIntegerPartition)

▷ `Total(IntegerPartition)` (attribute)

Returns: Integer

3.2.2 Parts (for IsIntegerPartition)

▷ `Parts(IntegerPartition)` (attribute)

Returns: List

3.2.3 Genus (for IsIntegerPartition)

▷ `Genus(IntegerPartition)` (attribute)

Returns: Integer

Calculates the Genus of the given integer partition,

3.2.4 Length (for IsIntegerPartition)

▷ `Length(IntegerPartition)` (attribute)

Returns: Integer

3.2.5 FrobeniusNumber (for IsIntegerPartition)

▷ `FrobeniusNumber(`*`IntegerPartition`*`)` (attribute)

Returns: Integer

3.2.6 Dual (for IsIntegerPartition)

▷ `Dual(`*`IntegerPartition`*`)` (attribute)

Returns: IntegerPartition

3.2.7 Dual2 (for IsIntegerPartition)

▷ `Dual2(`*`arg`*`)` (attribute)

3.2.8 IsSymmetric (for IsIntegerPartition)

▷ `IsSymmetric(`*`arg`*`)` (property)

Returns: `true` or `false`

3.2.9 IsPseudoSymmetric (for IsIntegerPartition)

▷ `IsPseudoSymmetric(`*`arg`*`)` (property)

Returns: `true` or `false`

3.2.10 IsPositiveSemiSymmetric (for IsIntegerPartition)

▷ `IsPositiveSemiSymmetric(`*`arg`*`)` (property)

Returns: `true` or `false`

3.2.11 IsNegativeSemiSymmetric (for IsIntegerPartition)

▷ `IsNegativeSemiSymmetric(`*`arg`*`)` (property)

Returns: `true` or `false`

3.2.12 IsSuperSemiSymmetric (for IsIntegerPartition)

▷ `IsSuperSemiSymmetric(`*`arg`*`)` (property)

Returns: `true` or `false`

3.2.13 IsAlmostSymmetric (for IsIntegerPartition)

▷ `IsAlmostSymmetric(`*`arg`*`)` (property)

Returns: `true` or `false`

3.2.14 Trace (for IsIntegerPartition)

▷ `Trace(`*`arg`*`)` (attribute)

3.2.15 Type (for IsIntegerPartition)

▷ `Type(arg)` (operation)

3.3 Integer SG Partitions

3.3.1 IsSGIntegerPartition (for IsIntegerPartition)

▷ `IsSGIntegerPartition(arg)` (property)

Returns: `true` or `false`

3.3.2 IsArf (for IsIntegerPartition)

▷ `IsArf(arg)` (property)

Returns: `true` or `false`

Chapter 4

Conversions

4.1 Numerical Sets and Integer Partitions

4.1.1 NumericalSet (for IsIntegerPartition)

▷ NumericalSet(*IntegerPartition*) (attribute)

Returns: NumericalSet

Calculates the numerical set correspondint to given integer partition.

4.1.2 IntegerPartition (for IsNumericalSet)

▷ IntegerPartition(*NumericalSet*) (attribute)

Returns: IntegerPartition

Calculates the integer partition correspondint to given numerical set.

Example
```
gap> S := NumericalSet( [0,3,4,5,7] );
{0,3,4,5,7,->}
gap> P := IntegerPartition( S );
    A - 1 1 6= 3+ 1+ 1+ 1
gap> S  = NumericalSet( P );
true
gap> Genus( S ) = Genus( P );
true
gap> Length( S ) = Length( P );
true
gap> FrobeniusNumber( S ) = FrobeniusNumber( P );
true
```

Example
```
gap> P := IntegerPartition( [7,2,1,1] );
SG  A + 4 2 11= 7+ 2+ 1+ 1
gap> P = IntegerPartition( NumericalSet( P ));
true
```

Chapter 5

nsip automatic generated documentation

5.1 nsip automatic generated documentation of attributes

5.1.1 DurfeeDecomposition (for IsIntegerPartition)

▷ `DurfeeDecomposition(`*arg*`)` (attribute)

5.1.2 Tree (for IsIntegerPartition)

▷ `Tree(`*arg*`)` (attribute)

Index